

Specifiche Sintattiche del Linguaggio Toy-Agent

Angelo Meo NF22500092

12 gennaio 2026

1 Il Programma

Il programma in Toy-Agent è strutturato rigidamente in due macro-sezioni sequenziali. A differenza di altri linguaggi, non è permesso mischiare dichiarazioni e logica.

1. **Blocco Memoria:** Dove vengono definite tutte le variabili.
2. **Blocco Codice:** Dove vengono definiti i **task** (funzioni/procedure).

Listing 1: Struttura Generale

```
Program ::= MemoryBlock CodeBlock
```

2 Le Dichiarazioni di Variabili (Memory Block)

Tutte le variabili utilizzate nel programma devono essere dichiarate all'interno del blocco **memory**: ... **end_memory**. Le dichiarazioni iniziano con la parola chiave **keep** e specificano il tipo con **as**.

Listing 2: Esempio di Dichiarazione

```
memory:
  keep contatore as whole;
  keep media as fract;
  keep is_valid as flag;
end_memory
```

Nota Semantica

Scope Unico (Global Scope):

Esiste una sola Tabella dei Simboli. Ogni identificatore dichiarato in **memory** è visibile in tutto il programma. È vietato dichiarare due variabili con lo stesso nome. Non esiste il concetto di "variabile locale" che nasconde una globale (shadowing).

2.1 Tipi di Dato

I tipi supportati sono rinominati rispetto agli standard per evitare bias semantici:

- **whole:** Numero intero (equiv. integer).
- **fract:** Numero in virgola mobile (equiv. float/real).
- **quote:** Stringa di testo (equiv. string).
- **flag:** Valore booleano **yes** o **no**.

3 I Task (Funzioni e Procedure)

La logica applicativa è suddivisa in unità chiamate **task**. Un task può accettare parametri in input e restituire un valore. La sintassi prevede la parola chiave **task**, seguita dal nome, la lista parametri tra parentesi quadre [...] e il tipo di ritorno (o **void** se omesso, ma qui assumiamo tipizzazione forte).

Listing 3: Sintassi Task

```
Task ::= "task" ID "[" Params "]" "->" Type ":" Body "done"
```

Nota Semantica

Gestione Parametri e Collisioni:

I parametri dichiarati nella firma del task (es. [x as whole]) vengono inseriti nella Tabella dei Simboli globale al momento della definizione. Per questo motivo, **i nomi dei parametri devono essere univoci** rispetto a tutte le variabili in memory e ai parametri di altri task.

3.1 Esempi Corretti ed Errati

Esempio Corretto:

```
memory:
    keep risultato as whole;
end_memory

task raddoppia [valore as whole] -> whole:
    % 'valore' e 'risultato' sono entrambi unici %
    yield valore times 2;
done
```

Esempio Errato (Collisione di Nomi):

```
memory:
    keep x as whole;
end_memory

task calcola [x as whole] -> whole: % ERRORE! %
    % 'x' e' gia' dichiarato in memory. %
    yield x plus 1;
done
```

4 Statement e Istruzioni

Il corpo di un task contiene una lista di istruzioni terminanti con punto e virgola.

4.1 Assegnazione

L'assegnazione utilizza l'operatore direzionale «.

```
ID "<<" Espressione;
```

Nota Semantica

Type Checking Rigido:

Non esiste inferenza o cast implicito. Non è possibile assegnare un valore **fract** (es. 5.5) a

una variabile dichiarata `whole`.

4.2 Controllo di Flusso (Check e Loop)

Al posto di *if* e *while*, Toy-Agent usa `check` e `loop`.

Condizionale (Check):

```
check (a over b) then
  max << a;
alt
  max << b;
close;
```

Iterazione (Loop):

```
loop (count under 10) do
  count << count plus 1;
close;
```

Nota Semantica

Vincolo Booleano:

Le espressioni usate in `check` e `loop` devono restituire tassativamente un tipo `flag` (`yes` o `no`). Espressioni aritmetiche (es. `if (1)`) provocano errore semantico.

5 Chiamata di Task e Return

5.1 Run (Call)

Per invocare un task si usa la keyword esplicita `run`.

```
risultato << somma run [10, 20];
```

5.2 Yield (Return)

Per restituire un valore si usa `yield`. Un task deve sempre contenere uno `yield` raggiungibile se dichiara un tipo di ritorno.

6 Specifiche Lessicali

Di seguito le mappature principali per l'analisi lessicale:

- **Operatori Aritmetici:** `plus (+)`, `minus (-)`, `times (*)`, `div (/)`.
- **Operatori Logici:** `and`, `or`, `not`.
- **Operatori Confronto:** `is (=)`, `is_not (<>)`, `over (>)`, `under (<)`.
- **I/O:** `show` (stampa a video), `grab` (lettura input).